

Pointfree CoEndoYoneda Lemma in Functional Categories Theory and Formalization in Lean 4

Luc Duponcheel Antigravity AI

July 26, 2026

Abstract

We present a variation of the CoYoneda Lemma for functional categories equipped with a monadic structure of its global elements, which we formally identify as *Functional Categories*. The central theoretical contribution, what we refer to as the *Pointfree CoEndoYoneda Equivalence*, establishes an isomorphism between, on the one hand, natural transformations mapping out of CoYoneda endofunctors to a designated kind of global endofunctors and, on the other hand, some corresponding kind of global elements. Our original motivation was pragmatic: formulate a CoEndoYoneda equivalence that could natively support both pure programs (functions) and impure programs (functions with side-effects, modeled via Kleisli categories). While this aspiration ultimately met a mathematical wall, the failure was profoundly revealing. The structural requirements of the *Functional Category* equivalence categorically reject arbitrary monadic effects. This limitation unexpectedly revealed a spectacular classification theorem: a naturality constraint of our framework mathematically forces the monad to be pure. We present a grand finale establishing a perfect 3-way mathematical equivalence between pointfree category theory and computational semantics (in casu idempotent monads). In addition to outlining the mathematical foundations, we detail the complete formalization of this result in the Lean 4 theorem prover. We discuss the technical challenges of working with functor compositions under strict definitional equality and demonstrate strategies for navigating dependent type theory to construct fully formal, pointfree proofs of higher-level category theory results.

1 Introduction

The *CoYoneda Lemma* serves as a cornerstone of modern category theory, asserting that for any category $\mathcal{C}$, object $X \in \mathcal{C}$, and functor $F : \mathcal{C} \rightarrow \mathbf{Set}$, natural transformations $\text{Nat}(\mathcal{C}(X, -), F)$ are isomorphic to functor evaluations $F(X)$. This result fundamentally states that an object X is completely determined by the morphisms out of it.

In computational settings, we frequently encounter categories equipped with additional structure. Categories that have a functor mapping functions of types (or sets) to programs, their images being called *pure programs*. We define such a category as a *Functional Category*. The world of computer programs, consists of both pure programs (without side-effects) *impure programs* (with side-effects). In this paper, we explore how a variation of the CoYoneda isomorphism extends in this setting. The variation replaces elements by global elements and is entirely pointfree. We also specialize this setting to Kleisli functions (monad-valued functions), as such focussing on *pure computations* versus *impure computations*, or, if you wish, *referential computation transparency* versus *non-referential computation transparency* (cfr. *referential evaluation transparency of expressions*).

Remarkably, we discover that the structural requirements of our CoYoneda equivalence serve as an airtight sieve for program purity, for programs that are Kleisli functions. In other words the structural requirements of our CoYoneda equivalence serve as an airtight sieve for computation purity (referential computation transparency).

We provide a comprehensive discussion of how these concepts and theorems are formalized in the Lean 4 proof assistant using the `Mathlib` library.

2 Functional Categories

Let $\mathcal{C}$ be a locally small category. We denote the category of types (or sets) as **Type**. For any object $X \in \mathcal{C}$, the covariant Hom-functor, which we will call the CoYoneda functor, is defined as:

$$CYF_X : \mathcal{C} \rightarrow \mathbf{Type}, \quad Y \mapsto \mathcal{C}(X, Y)$$

Definition 2.1. A *Functional Category* is a category $\mathcal{C}$ equipped with the following data:

1. A functor $\Phi : \mathbf{Type} \rightarrow \mathcal{C}$.
2. An endofunctor $GF : \mathcal{C} \rightarrow \mathcal{C}$ defined by $GF(X) = \Phi(CYF_{\Phi(1)}(X))$.
3. A monadic structure on the globals of $\mathcal{C}$, provided by
 - (a) a unit natural transformation $\gamma\eta : \text{Id}_{\mathcal{C}} \rightarrow GF$
 - (b) a multiplication natural transformation $\gamma\mu : GF \circ GF \rightarrow GF$.

These structural components must satisfy the monad left unit law:

$$\gamma\eta_{GF(X)} \circ \gamma\mu_X = id_{GF(X)}$$

For now no other extra monad laws are needed in the formal Lean 4 proof. This is an interesting observation on its own.

The structure above essentially allows $\mathcal{C}$ to reflect the category of types into itself, while treating GF as a monadic context. We define the *CoYoneda Endofunctor* $CYEF_X : \mathcal{C} \rightarrow \mathcal{C}$ for a fixed object X as the composition:

$$CYEF_X = \Phi \circ CYF_X$$

We use the following definition of *global element* as a morphisms from the canonical singleton $\Phi(1)$ to X .

$$G_\Phi(X) = \mathcal{C}(\Phi(1), X)$$

2.1 Formalization of Functional Categories in Lean 4

In Lean 4, a functional category is modeled as a type class extending the standard `LargeCategory` from `Mathlib`.

```

abbrev CYF {C : Type (u + 1)} [LargeCategory.{u} C] (X : C) : C → Type u :=
  coyoneda.obj (op X)

abbrev CYEF_def
  {C : Type (u + 1)} [LargeCategory.{u} C] (Φ : Type u → C) (X : C) : C → C :=
  CYF X >>> Φ

abbrev GEF_def {C : Type (u + 1)} [LargeCategory.{u} C] (Φ : Type u → C) : C → C :=
  CYEF_def Φ (Φ.obj PUnit)

class FunctionalCategory (C : Type (u + 1)) extends LargeCategory.{u} C where
  Φ : Type u → C

  γμ : (GEF_def Φ >>> GEF_def Φ) → GEF_def Φ
  γη : 1 → GEF_def Φ

  γ_left_unit :
    ∀ (X : C), γη.app ((GEF_def Φ).obj X) >> γμ.app X = id ((GEF_def Φ).obj X)

  φ {X Y : Type u} (f : X → Y) : Φ.obj X → Φ.obj Y := Φ.map (TypeCat.ofHom f)

  φ_eq :
    ∀ {X Y : Type u} (f : X → Y), φ f = Φ.map (TypeCat.ofHom f) :=
    by intros; rfl

  γη_φ :
    ∀ (X : Type u), γη.app (Φ.obj X) = φ (fun x => φ (fun _ => x)) :=
    by intros; rfl

```

and global elements are modeled as

```

abbrev G
  {C : Type (u + 1)} [LargeCategory.{u} C] (Φ : Type u → C) (X : C) : Type u :=
  Φ.obj PUnit → X

```

A canonical, trivial instance of the type class is **Type** with Φ as the identity functor, in other words, the world of pure programs without side-effects (functions).

```

instance typesFunctionalCategory : FunctionalCategory (Type u) where
  Φ := 1 (Type u)
  γη := {
    app := fun X ⇒ (1 (Type u)).map (λ(fun x ⇒ λ(fun _ ⇒ x)))
    naturality := fun X Y f ⇒ by ext; rfl
  }
  γμ := {
    app := fun X ⇒ (1 (Type u)).map (λ(fun (f : PUnit → PUnit → X) ⇒ λ(fun pu ⇒ f pu
    pu)))
    naturality := fun X Y f ⇒ by ext; rfl
  }
  γ_left_unit := fun _ ⇒ by rfl

```

3 The CoEndoYoneda Equivalence

The requirements of `FunctionalCategory` turn out to be sufficient to establish a CoEndoYoneda equivalence for endofunctors.

3.1 Auxiliary definitions and theorems

We construct a designated global element from a designated natural transformation as follows

```

def τx2ggfx {F : C ⇔ C} {X : C} (τx : CYEF X → F ≫ GEF) :
  Global ((F ≫ GEF).obj X) :=
  let ggfx := φ (fun _ ⇒ id X) >> τx.app X
  ggfx

```

We construct a designated (natural, not shown) transformation from a designated global element as follows

```

def ggfx2τx
  {F : C ⇔ C} {X : C} (ggfx : Global (GEF.obj (F.obj X))) : CYEF X → (F ≫ GEF) :=
  let τx : CYEF X → F ≫ GEF := {
    app Y := φ (ggfx >> (F ≫ GEF).map .) >> γμ.app (F.obj Y)
    naturality _ _ h := ...
  }
  τx

```

We formulate (and prove, not shown) a left inverse theorem as follows

```

theorem left_inverse {F : C ⇔ C} {X : C} (τx : CYEF X → (F ≫ GEF)) :
  τx = ggfx2τx (τx2ggfx τx) := by ...

```

We formulate (and prove, not shown) a right inverse theorem as follows

```

theorem right_inverse {F : C ⇔ C} {X : C} (ggfx : Global ((F ≫ GEF).obj X)) :
  ggfx = τx2ggfx (ggfx2τx ggfx) := by ...

```

Notice that both inverses apply φ (defined as $\Phi.map$) to $\text{fun } _ \Rightarrow \text{id}_X$, which mirrors the classic CoYoneda construction.

3.2 CoEndoYoneda Equivalence

Utilizing the auxiliary definitions and theorems above we can now define the main equivalence.

```

def coEndoYonedaEquiv {F : C → C} {X : C} :
  (C YEF X → (F >>> GEF)) ≈ Global ((F >>> GEF).obj X) where
toFun      := τx2ggfx
invFun     := ggfx2τx
left_inv   := fun τx => (left_inverse τx).symm
right_inv  := fun τx => (right_inverse τx).symm

```

This equivalence provides a completely pointfree and formally verified method for evaluating and constructing natural transformations in functional categories.

4 The Essence of Purity (Every disadvantage has an advantage)

4.1 The Impure Aspiration

The original motivation for this research was entirely pragmatic: we sought to define a CoEndoYoneda equivalence that could play a role in the programming world, which encompasses both pure programs (functions) and impure programs (computations with side-effects, such as state mutation or I/O). A natural setting for such programs is the Kleisli category of a Monad M , denoted `KleisliCat M`.

We embarked on attempting to instantiate `FunctionalCategory` using this `KleisliCat M` over `Type` to construct an equivalence that could natively handle side-effects. However, this attempt revealed an apparent limitation: the equivalence cannot be instantiated for arbitrary monads. Our `FunctionalCategory` definition, while structurally elegant, strictly rejected standard computational effects.

But this disadvantage birthed a profound mathematical advantage: one of the `FunctionalCategory` requirements acts as a classification of purity for `KleisliCat M`.

It declares referential computation transparency in a categorical way.

4.2 The Purity Sieve

The core constraint responsible for this resistance is $\gamma\eta$ -naturality. When evaluated in `KleisliCat M`, this naturality requirement imposes that mapping `pure` over the structure of a monadic value mx is exactly the same as returning the value mx *without* triggering any side-effects. Formally, it enforces the computational equality:

$$mx \gg= \lambda x. \text{pure}(\text{pure}(x)) = \text{pure}(mx)$$

We define this precisely in Lean as `EnforcesPurity`:

```

def EnforcesPurity : Prop :=
  ∀ {X : Type} {v} (mx : M X), (mx >>= fun x => pure (pure x)) = pure mx

```

One immediate consequence of **EnforcesPurity** is that the monad’s shape functor evaluated at the terminal object **PUnit** must be trivial (proof not shown):

```
theorem purity_implies_subsingleton_shape
  (h-pure : EnforcesPurity M) : Subsingleton (M PUnit) := by ...
```

Intuitively, **Subsingleton (M PUnit)** demands that the monad cannot express any meaningful impureness at the unit level. For example, for the **State Monad** $(S \rightarrow (Y \times S))$, evaluating at **PUnit** yields $S \rightarrow (\mathbf{1} \times S) \cong S \rightarrow S$. The only way $S \rightarrow S$ can be a subsingleton is if S itself is a subsingleton (i.e., there is no state). Thus, State is successfully rejected.

However, **Subsingleton (M PUnit)** is a necessary but *insufficient* condition for complete purity. Consider the **Reader Monad** $(R \rightarrow Y)$. At **PUnit**, this is $R \rightarrow \mathbf{1}$, which is *always* a subsingleton regardless of what the environment R is, meaning the Reader Monad slips through this partial net.

But our **EnforcesPurity** constraint is far more robust. When applied to the Reader Monad over a non-trivial environment (like **Bool**), the equation

$$(m \gg= \lambda x. \text{pure}(\text{pure}(x))) = \text{pure}(m)$$

fails, filtering out the Reader Monad as well.

4.3 A Universal 3-Way Equivalence

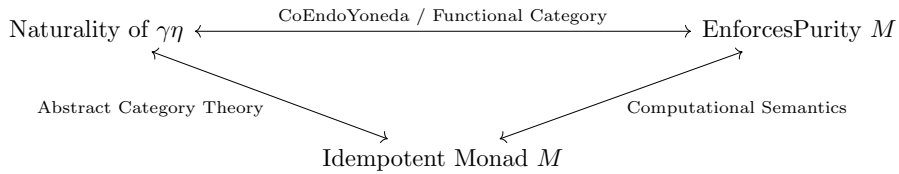
If **EnforcesPurity** filters out every non-trivial computational effect, what does it mean mathematically? By expanding the monadic **bind**, the definitional equation required for purity simplifies seamlessly to:

$$\text{fmap pure } m = \text{pure } m$$

In categorical terms, this is precisely the requirement $T\eta = \eta T$, which is one of the defining characteristics of an **Idempotent Monad** [2, 1].

An idempotent monad represents a computational context where layering the effect twice adds no new information over applying it once. In the category of Sets, it is a well-known theorem that all idempotent monads correspond to full reflective subcategories [3, 1], effectively forcing the monad to not add anything.

We formalized this profound connection in Lean 4, establishing a complete, closed-loop 3-way equivalence between *pointfree* Category Theory morphisms (the $\gamma\eta$ -naturality of our Functional Category), Computational Semantics (**EnforcesPurity**), and *pointful* Category Theory monad-valued functions (**IdempotentMonad**).



We completely verified this loop in Lean. The forward direction utilizes `bind_pure_comp` to collapse the naturality square into purity, while the reverse direction beautifully relies on `bind_assoc` to inflate the purity constraint back into the full categorical square.

```

theorem enforces_purity_implies_naturality
  (h_pure : EnforcesPurity M)
  (γη_app : (Z : KleisliCat M) → (Z → (GEF_def (κΦ M)).obj Z))
  (h_γη_app : ∀ (Z : KleisliCat M), γη_app Z =
    show Z → M (PUnit.{v+1} → M Z) from fun z => pure (fun _ : PUnit.{v+1} => pure z))
  :
  ∀ {X Y : KleisliCat M} (f : X → Y),
  f >> γη_app Y = γη_app X >> (GEF_def (κΦ M)).map f := by ...

```

Thus, we confirm our ultimate thesis

Referential transparency is exactly the requirement for a computational context to admit a global coyoneda endofunctor equivalence.

5 Proof Techniques and Challenges in Lean 4

Formalizing abstract category theory in dependent type theory (DTT) introduces unique difficulties, for example in `left_inverse`, particularly concerning definitional equality.

5.1 Strict Functor Composition

In standard mathematics, functor composition is strictly associative. In Lean 4, however, these are represented as distinct abstract syntax trees. They are *isomorphic* via `Category.assoc`, but they are not *definitionally equal*.

When writing the proof of `left_inverse`, this limitation prevents standard term-rewriting tactics like `simp` from effortlessly reducing deeply nested compositions.

5.2 Constructing Equality Chains

To circumvent the strict type-checker without resorting to manual casts (such as `eqToHom`), the proof for `left_inverse` abandons the monolithic `simp` approach in favor of explicit equality chains acting on the natural transformation components.

By utilizing `Eq.trans` and `congr_arg`, we explicitly guide the type checker through the unit laws and naturality squares:

```

have h1 :
  τx.app Y = τx.app Y >> id ((GEF_def Φ).obj (F.obj Y)) :=
  (Category.comp_id (τx.app Y)).symm
have h2 :
  id ((GEF_def Φ).obj (F.obj Y)) =
  γη.app ((GEF_def Φ).obj (F.obj Y)) >> γμ.app (F.obj Y) :=
  (γ_left_unit (F.obj Y)).symm
have h_left_unit :
  τx.app Y =
  τx.app Y >> (γη.app ((GEF_def Φ).obj (F.obj Y)) >> γμ.app (F.obj Y)) :=
  Eq.trans h1 (congr_arg (τx.app Y >> .) h2)

```

This localized approach bypasses the rigid typing of the category morphisms by acting directly on the equality type `=` in the universe of types. This technique is highly robust and avoids the infamous “motive is not type correct” errors common in Lean category theory formalization. Defeating type mismatches often requires subtle casting nudges, such as `show ... from ...`, to force Lean to unify deep definitional equalities within higher-order functions.

6 Conclusion

The Pointfree CoEndoYoneda Lemma is a variation of the classical Pointful CoYoneda Lemma in a functional category setting. While our initial aim was to generalize this equivalence to arbitrary computational effects via monads, the rigorous formalization process revealed that the required categorical structure fundamentally filters out impurities.

By successfully formalizing the lemma in Lean 4, we have not only verified its absolute formal correctness but also established that `FunctionalCategory` mathematically classifies referential transparency, uncovering a 3-way mathematical equivalence spanning pointfree category theory and pointful computational semantics.

Moreover, we have demonstrated a concrete methodology for handling strict composition equality in dependent type theory. The approach of combining explicit structural definitions with targeted, step-by-step equality chaining provides a reliable method for formalizing advanced category theory in modern proof assistants.

References

- [1] Francis Borceux, *Handbook of Categorical Algebra* (Volumes 1–3). Encyclopedia of Mathematics and its Applications. Cambridge University Press, 1994.
- [2] nLab authors, *Idempotent monad*, <https://ncatlab.org/nlab/show/idempotent+monad>.
- [3] nLab authors, *Reflective subcategory*, <https://ncatlab.org/nlab/show/reflective+subcategory>.